

# BOSS HUNTER MMO RPG

작성자 : 안승현

# 목차

- Boss Hunter 소개
- Mirror 소개
- 계획
  - 1주차
  - 2주차
  - 3주차
  - 4주차
- 개발에 사용된 에셋
- 구현에 어려웠던 점
- 해결
- 개발 후 느낀점 혹은 추후 개선 방향

# BOSS HUNTER 소개

- UNITY ENGINE (2021.3.19f) 사용
- URP(Universal Render PipeLine)라는 랜더링 방식 채택
- RPG 게임의 기본적인 요소 구현 ex) 인벤토리, 스킬, 강화, 파티플레이 등등
- Mirror API를 통한 클라이언트, 호스트, 서버 구현
- 유료 에셋 및 무료 에셋 사용
- 최대 100명까지의 멀티플레이 지원
- 실시간 네트워크 서비스
- <https://www.youtube.com/watch?v=ubSgPd6OHsY> (YouTube 영상)



# MIRROR API 소개

- Unet으로부터 포크(fork)해 다수의 개발자들이 오픈 소스로 제작한 Mirror
- 무료 오픈 소스 게임 네트워킹 라이브러리 입니다.
- 누구나 쉽게 네트워킹 게임을 제작할 수 있는 에셋 입니다.

Mirror 공식 다운로드 사이트

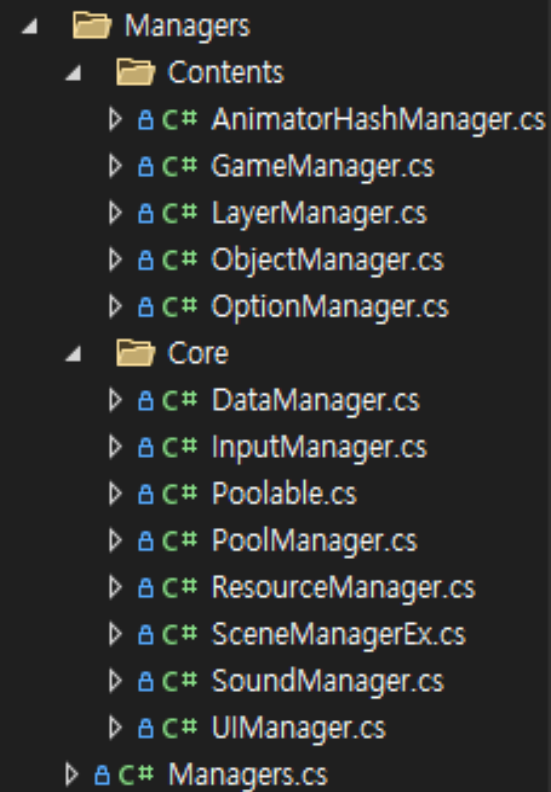
<https://assetstore.unity.com/packages/tools/network/mirror-129321?locale=ko-KR>

# MIRROR API 특징

1. Mirror는 Unet에서 포크되었기 때문에 마찬가지로 HLAPI 입니다. Mirror를 사용하는 개발자들은 서버 엔진 단의 작동 방식을 몰라도 API만 잘 활용하면 손쉽게 멀티플레이를 구현할 수 있습니다.
2. Mirror는 초기에 TCP 베이스로 탄생했으나 게임에 맞는 다른 Transport를 사용하면 자동으로 성능 최적화가 가능합니다.
3. Server/Client의 서버 모델을 지원합니다. 데디케이티드 서버(Dedicated Server), 권한 서버 (Authority Server) 라고도 하는 이 방식은 실제로 게임 플레이어의 핵심 로직을 담당하는 전용 서버 + 클라이언트로 구성됩니다. 특히 경쟁 요소가 있는 게임들에는 보안 이슈를 다루는 것이 무척이나 중요한데, 이를 서버 쪽에서 처리할 수 있게 됩니다. 추가로 리슨 서버(Listen Server) 방식도 지원하여 한 명의 플레이어가 서버 역할을 대신하고 호스트(방장)가 되는 것도 가능합니다.
4. 네트워크의 전반적인 기능은 NetworkBehaviour라는 상위 오브젝트에 구현된 메서드를 override해서 구현합니다.
5. 오픈 소스 기반 라이브러리입니다. 다수의 개발자들이 꾸준히 Github에 업데이트를 하고 있으며 많은 온라인 게임 프로젝트들이 Mirror로 개발 & 서비스되고 있습니다. Mirror 전용 Discord 채널에는 개발 내용을 공유하고 질문을 할 수 있는 커뮤니티가 잘 구축되어 있습니다.
6. 오랜 기간 개발되어 API 문서와 튜토리얼이 잘 되어있습니다.

# 1주차 – 구조 설계

- Object Pooling(오브젝트 풀링), New Input(새로운 인풋 시스템), addressable(에셋 로드), Data(데이터), Sound(소리), UI(사용자 인터페이스) 등등 자주 사용하는 클래스를 만들고 싱글톤으로 접근하기 쉽게 설계
- Mirror Api 구조 파악 후 오프라인, 온라인 씬으로 나누고 온라인 씬에서 필요한 씬들은 동적으로 씬을 로드하는 방식으로 구현 예정
- 플레이어 즉 클라이언트는 하나의 오브젝트만 소유하고 조종할 수 있다는 전제하에 코드 작성
- 닉네임 + 원하는 캐릭터 선택시 캐릭터가 생성되고 마을에서 시작하도록 설계



# 1주차 – 구조 설계

```
public class Managers : NetworkManager
{
    static Managers instacne;
    public static Managers Instance
    {
        get
        {
            Init();
            return instacne;
        }
    }
}

#region Contents
AnimatorHashManager _animHash = new AnimatorHashManager();
GameManager _game = new GameManager();
LayerManager _layerManager = new LayerManager();
ObjectManager _object = new ObjectManager();
OptionManager _option = new OptionManager();

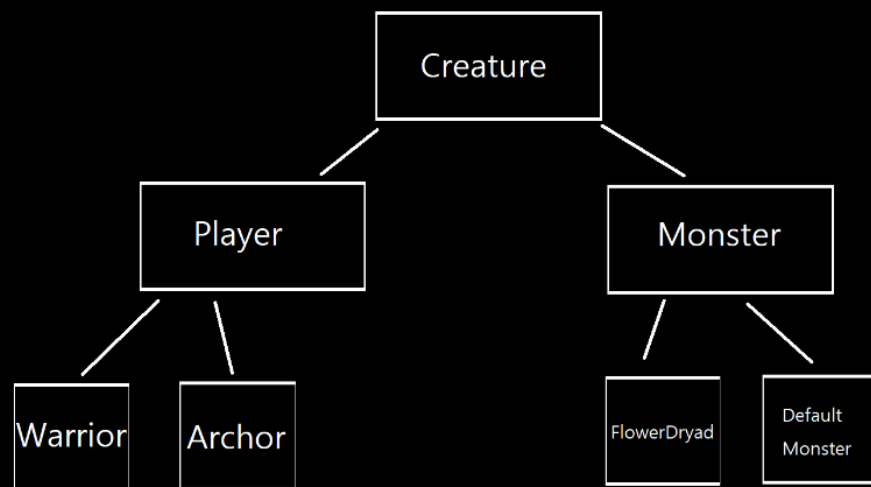
public static AnimatorHashManager AnimHash { get { return Instance?._animHash; } }
public static LayerManager LayerManager { get { return Instance?._layerManager; } }
public static GameManager Game { get { return Instance?._game; } }
public static OptionManager Option { get { return Instance?._option; } }
public static ObjectManager Object { get { return Instance?._object; } }
#endregion

#region Core
DataManager _data = new DataManager();
InputManager _input = new InputManager();
PoolManager _pool = new PoolManager();
ResourceManager _resource = new ResourceManager();
SceneManagerEx _scene = new SceneManagerEx();
SoundManager _sound = new SoundManager();
UIManager _ui = new UIManager();

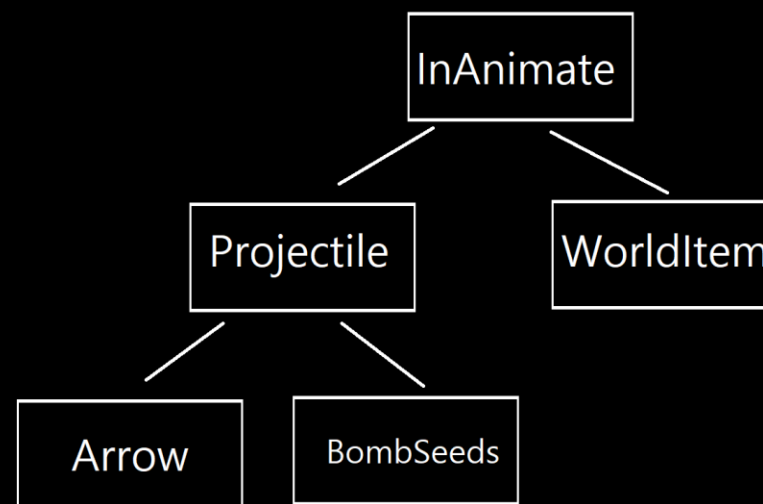
public static DataManager Data { get { return Instance?._data; } }
public static InputManager Input { get { return Instance?._input; } }
public static PoolManager Pool { get { return Instance?._pool; } }
public static ResourceManager Resource { get { return Instance?._resource; } }
public static SceneManagerEx Scene { get { return Instance?._scene; } }
public static SoundManager Sound { get { return Instance?._sound; } }
public static UIManager UI { get { return Instance?._ui; } }
#endregion
```

- 매니저 클래스
  - 싱글톤으로 구현하여 인스턴스 생성 비용 절감 및 접근성 향상
  - 매니저를 관리하는 매니저 클래스
  - 각 클래스의 기능을 나눠서 static 변수로 정의

# 1주차 - 구조 설계

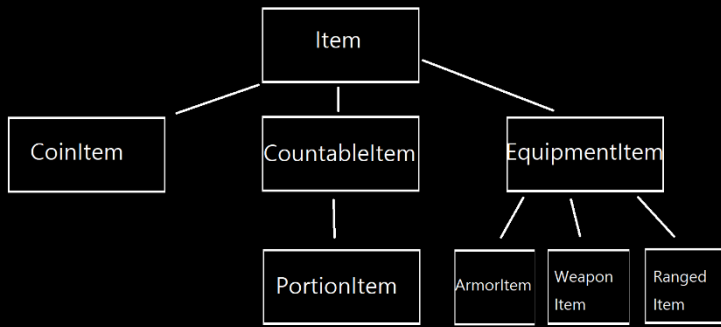


생명체 클래스 구조

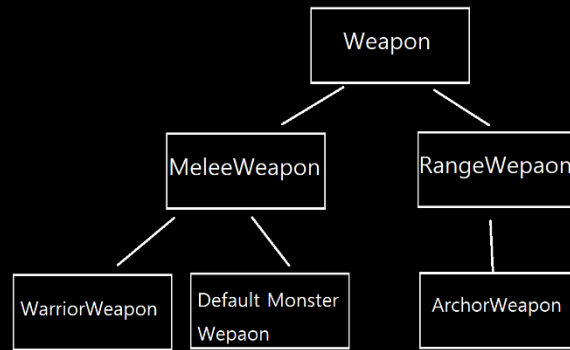


비생명체 클래스 구조

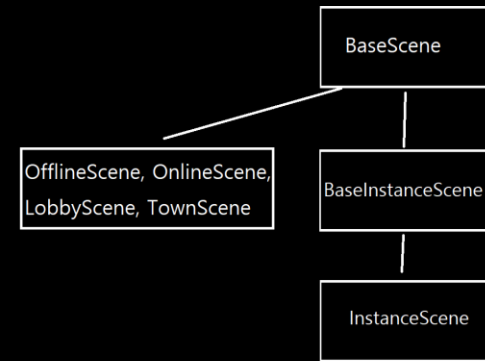
# 1주차 - 구조 설계



아이템 클래스 구조



무기 클래스 구조



게임 씬 클래스 구조

## 2주차 – 플레이어 몬스터 및 기능 구현

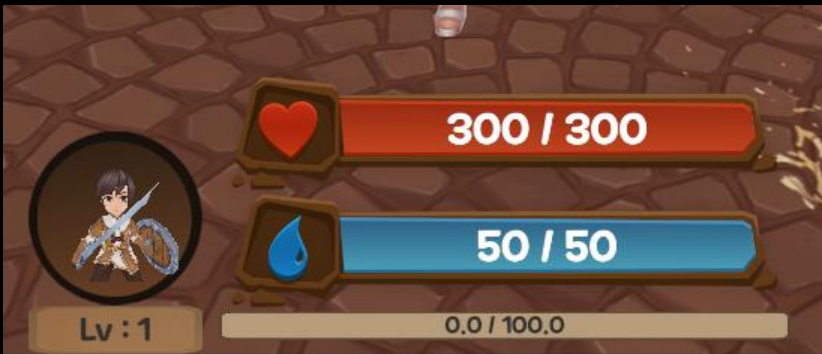
- 플레이어를 주시할 카메라 스크립트 작성  
단 시네머신 기반으로 스크립트를 작성하여 기능을 확장할 예정
- 플레이어의 구성요소를 Player 클래스에 Stat, StateMachine, PlayerController 등 각 기능을 세분화하여 구현할 예정
- 마을, 채팅, 인스턴스 던전, 플레이어UI, 몬스터 구현 예정

```
Stat
├── C# MonsterStat.cs
├── C# PlayerStat.cs
└── C# Stat.cs
```

```
Controllers
├── C# BaseController.cs
├── C# CameraController.cs
├── C# MonsterController.cs
└── C# PlayerController.cs
```

```
StateMachine
├── BaseState
│   ├── MonsterState
│   │   ├── FlowerDryad
│   │   │   └── C# MonsterDissemination.cs
│   │   ├── C# BaseStateMonster.cs
│   │   ├── C# MonsterAttackState.cs
│   │   ├── C# MonsterChaseState.cs
│   │   ├── C# MonsterIdleState.cs
│   │   ├── C# MonsterPatrolState.cs
│   │   └── C# MonsterReturnState.cs
│   └── PlayerState
│       ├── C# AttackState.cs
│       ├── C# BaseStatePlayer.cs
│       ├── C# DashState.cs
│       ├── C# IdleState.cs
│       ├── C# JumpState.cs
│       ├── C# MoveState.cs
│       ├── C# OnWallState.cs
│       ├── C# ProjectileAttackState.cs
│       └── C# RunState.cs
├── C# BaseState.cs
├── C# DeadState.cs
├── C# DeadStatePhysics.cs
├── C# HitState.cs
├── C# MonsterStateMachine.cs
├── C# PlayerStateMachine.cs
└── C# StateMachine.cs
```

## 2주차 - 플레이어 몬스터 및 기능 구현



마을  
인스턴스 던전  
플레이어 정보 UI  
채팅창 구현

# 2주차 – 플레이어 몬스터 및 기능 구현

```
protected override void InitStateMachine()
{
    IdleState idleState = new IdleState(Define.State.Idle, this, controller);
    stateMachine.DefaultState(Define.State.Idle, idleState);

    MoveState moveState = new MoveState(Define.State.Moving, this, controller);
    stateMachine.RegisterState(Define.State.Moving, moveState);

    JumpState jumpState = new JumpState(Define.State.Jumping, this, controller);
    stateMachine.RegisterState(Define.State.Jumping, jumpState);

    DashState dashState = new DashState(Define.State.Dash, this, controller, 3, 3, 1, 1);
    stateMachine.RegisterState(Define.State.Dash, dashState);

    RunState runState = new RunState(Define.State.Run, this, controller);
    stateMachine.RegisterState(Define.State.Run, runState);

    AttackState attackState = new AttackState(Define.State.NormalAttack, this, controller);
    stateMachine.RegisterState(Define.State.NormalAttack, attackState);

    HitState hitState = new HitState(Define.State.Hit, this, controller);
    stateMachine.RegisterState(Define.State.Hit, hitState);

    DeadState deshState = new DeadState(Define.State.Dead, this, controller);
    stateMachine.RegisterState(Define.State.Dead, deshState);

    OnWallState onWallState = new OnWallState(Define.State.OnWall, this, controller);
    stateMachine.RegisterState(Define.State.OnWall, onWallState);
}
```

## 플레이어

- 플레이어의 기능을 FSM으로 구현하여 Creature->Player->Warriror로 구성된 클래스에서 InitStateMachine()을 오버라이드하여 정의 하였습니다.

# 2주차 – 플레이어 몬스터 및 기능 구현

```
public override bool CheckCondition()
// 추가적인 점프키 누르지 않았으면 스킵
if (PlayerControllerGS.IsJump == false)
    return false;

Vector3 rayPos = creatureGS.transform.position;
float maxDistance = 1.5f;
// 콜라이더 높이 인터페이스가 있으면 대응 순 높이 중 y축 배치
if (creatureGS is IColliderInfo)
{
    IColliderInfo colliderHeight = creatureGS as IColliderInfo;
    rayPos.Set(rayPos.x, rayPos.y + colliderHeight.GetHeight() * 0.75f, rayPos.z);
    maxDistance = colliderHeight.GetRadius() + 0.5f;
}

// 점프중이고 점프키를 눌렀다면 Ray를 쏘서 벽이 있는지 확인한다.
return creatureGS.GetSetPhysics.Raycast(rayPos, creatureGS.transform.forward, out hit, maxDistance, Managers.LayerManager.Wall, QueryTriggerInteraction.Ignore);
}

public override void EnterState()
{
    PlayerGS.GetNetAnim.Animator.SetBool(Managers.AnimHash.BJumpIn, false);
    PlayerGS.GetNetAnim.Animator.SetBool(Managers.AnimHash.BJumpOut, false);

    // 콜라이더 높이 인터페이스가 있으면 대응 순 높이 중 y축 배치 콜라이더 반지를 만들었으면
    Vector3 destPos = hit.point;
    if (creatureGS is IColliderInfo)
    {
        IColliderInfo colliderInfo = creatureGS as IColliderInfo;
        // 만약 Ray캐스트가 성공적으로 되면 Ray위치로 플레이어 이동
        destPos.Set(hit.point.x - colliderInfo.GetRadius(), hit.point.y - colliderInfo.GetHeight() * 0.75f, hit.point.z - colliderInfo.GetRadius());
    }

    // 만약 Ray캐스트가 성공적으로 되면 Ray위치로 플레이어 이동
    creatureGS.GetRigidBody.MovePosition(destPos);
    // 벽으로 보도록 이동시킨다.
    creatureGS.GetRigidBody.MoveRotation(Quaternion.LookRotation(-hit.normal));

    // 충격 끝다.
    creatureGS.GetRigidBody.useGravity = false;
    // 벽타기 애니메이션 재생
    creatureGS.GetNetAnim.Animator.SetBool(Managers.AnimHash.BOnWall, true);
    // 컨트롤러에서 충격 제어 X
    PlayerControllerGS.IsOnWall = true;
}
```

## 플레이어의 점프 상태 코드

- 플레이어가 점프 상태에서 해당 오브젝트의 Layer가 Wall인 경우에 스페이스 버튼을 누르면 해당 상태로 진입합니다. 해당 플레이어의 콜라이더의 높이의 0.75배에서 전방으로 Raycast를 발사하여 오브젝트를 추적하고 만약 충돌되었으면 콜라이더의 높이의 0.75배를 다시 빼고 해당 위치로 플레이어의 이동시키고 충격을 비활성화로 구현하였습니다.

## 2주차 – 플레이어 몬스터 및 기능 구현

```
[ServerCallback]
protected override void InitStateMachine()
{
    MonsterIdleState m_idleState = new MonsterIdleState(Define.State.Idle, this, controller);
    stateMachine.DefaultState(Define.State.Idle, m_idleState);

    MonsterChaseState m_chaseState = new MonsterChaseState(Define.State.Chase, this, controller);
    stateMachine.RegisterState(Define.State.Chase, m_chaseState);

    MonsterReturnState m_returnState = new MonsterReturnState(Define.State.Return, this, controller);
    stateMachine.RegisterState(Define.State.Return, m_returnState);

    MonsterPatrolState m_patrolState = new MonsterPatrolState(Define.State.Patrol, this, controller);
    stateMachine.RegisterState(Define.State.Patrol, m_patrolState);

    MonsterAttackState m_attackState = new MonsterAttackState(Define.State.NormalAttack, this, controller);
    stateMachine.RegisterState(Define.State.NormalAttack, m_attackState);

    HitState m_hitState = new HitState(Define.State.Hit, this, controller);
    stateMachine.RegisterState(Define.State.Hit, m_hitState);

    DeadState m_deadState = new DeadState(Define.State.Dead, this, controller);
    stateMachine.RegisterState(Define.State.Dead, m_deadState);
}
```

몬스터 인공지능(FSM 상태머신)

- 몬스터도 플레이어와 마찬가지로 FSM으로 행동 패턴을 정의하였습니다.
- 각 상태마다 Idle 상태가 정의되어 있으며 이 상태를 기점으로 상태로 전환이 가능하도록 코드가 작성되어 있습니다.
- 각 상태마다 조건이 있어 해당 조건을 만족해야 상태가 전환이 됩니다.

## 2주차 – 플레이어 몬스터 및 기능 구현

```
public interface IHitable
{
    public bool IsAlive { get; }

    public void OnAttacked(Stat attacker, int hitpriority = 0);

    public void OnAttacked(int _damage, int hitpriority = 0);

    public void OnAttacked(Stat attackerOwner, int damage, int hitpriority = 0);
}
```

```
public interface IAttackEvent
{
    [ServerCallback]
    public abstract void OnStartAttack();

    [ServerCallback]
    public abstract void OnEndAttack();
}
```

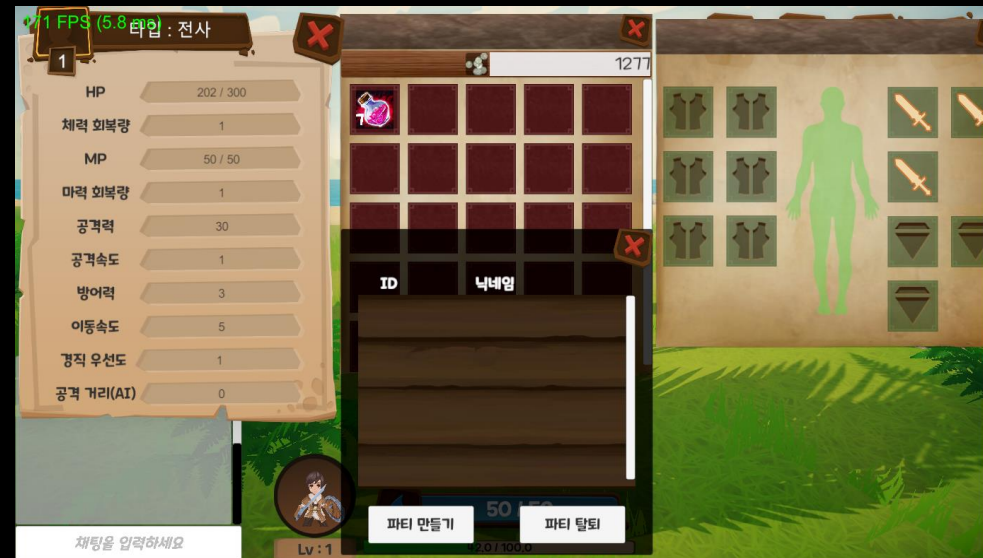
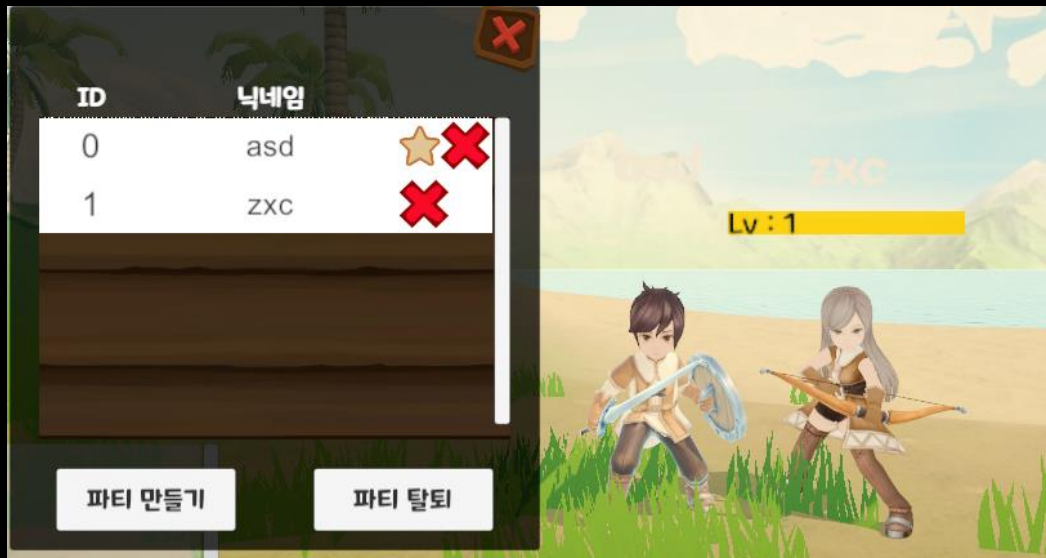
### 인터페이스

- 인터페이스를 정의하여 좀 더 유연하게 확장성 있게 코드를 작성 할 수 있습니다.
- 각 기능을 다양하게 구현이 가능함
- 공격했을 때 해당 인터페이스의 상속 여부만 확인하여 호출하거나 하지 않으면 된다.
- 해당 기능을 구현하여 사용하고 싶으면 직접 이 인터페이스를 상속받아 기능을 구현

Ex) 몬스터 피격, NPC는 논 피격

# 3주차 – 인스턴스 던전 및 아이템 구현

- 플레이어는 인스턴스 맵 을 통해 던전으로 들어갈 수 있다. 각 던전에서 일반 몬스터, 보스 몬스터 존재 보스 몬스터가 제거되면 해당 인스턴스 던전이 종료되고 사라진다
- 파티플레이 구현 파티를 통해 같이 던전을 탐험
- MMORPG의 필수적인 아이템 장비창과 플레이어 스탯창 구현



# 3주차 – 인스턴스 던전 및 아이템 구현

```
[Serializable]
public abstract class ItemData
{
    public int _id;
    public Define.ItemType _itemType;
    public string _name;
    public string _tooltip;
    public string _iconSprite;
    public string _dropItemPrefab;

    // Ä,ÄÖ; ,ÄÄ K&ÄÄÜ »Ý&e
    public abstract Item CreateItem();
}

[Serializable]
public class CountableItemData : ItemData
{
    public int _maxAmount = 99;

    public override Item CreateItem()
    {
        return new CountableItem(this);
    }
}

[Serializable]
public class PortionItemData : CountableItemData
{
    public Define.EffectType _effectType;
    public float _value;

    public override Item CreateItem()
    {
        return new PortionItem(this);
    }
}
```

```
[Serializable]
public class ArmorItemData : ILoader<int, ArmorItemData>
{
    public List<ArmorItemData> items = new List<ArmorItemData>();

    public Dictionary<int, ArmorItemData> MakeDict()
    {
        Dictionary<int, ArmorItemData> dict = new Dictionary<int, ArmorItemData>();
        foreach (ArmorItemData item in items)
            dict.Add(item._id, item);
        return dict;
    }
}

[Serializable]
public class WeaponItemData : EquipmentItemData
{
    public int Damage => _damage;
    private int _damage;

    public override Item CreateItem()
    {
        return null;
        //return new WeaponItem(this);
    }
}

[Serializable]
public class WeaponItemData : ILoader<int, WeaponItemData>
{
    public List<WeaponItemData> items = new List<WeaponItemData>();

    public Dictionary<int, WeaponItemData> MakeDict()
    {
        Dictionary<int, WeaponItemData> dict = new Dictionary<int, WeaponItemData>();
        foreach (WeaponItemData item in items)
            dict.Add(item._id, item);
        return dict;
    }
}
```

## 데이터

- 플레이어, 몬스터, 아이템, 등 데이터를 모두 JSON으로 정의한 후 게임 실행시 데이터를 역직렬화해서 DataManager의 각 Dictionary에 저장하도록 설계

# 3주차 – 인스턴스 던전 및 아이템 구현

```
protected Dictionary<uint, GameObject> networkPlayerObjects = new Dictionary<uint, GameObject>();  
protected Dictionary<uint, GameObject> networkMonsterObjects = new Dictionary<uint, GameObject>();  
protected Dictionary<uint, GameObject> networkInAnimateObjects = new Dictionary<uint, GameObject>();
```

```
[ServerCallback]  
public virtual void S_SceneObjectRemove()  
{  
    if (networkPlayerObjects.Count == 0)  
    {  
        // 씬 텍세처에서 제거  
        Managers.Scene.RemoveBaseScene(gameObject.scene);  
  
        // 스폰너 기능 중지  
        foreach (MonsterSpawner monsterSpawner in monsterSpawners)  
        {  
            monsterSpawner.StopAllCoroutines();  
        }  
  
        // 몬스터 제거  
        foreach (GameObject monster in networkMonsterObjects.Values)  
        {  
            NetworkServer.UnSpawn(monster);  
        }  
  
        // 기타 오브젝트 제거  
        foreach (GameObject item in networkInAnimateObjects.Values)  
        {  
            NetworkServer.UnSpawn(item);  
        }  
  
        Invoke("DelayUnloadScene", 3.0f);  
    }  
}
```

## • 던전 씬 생명주기

- BaseScene이 현재 씬에 있는 모든 오브젝트를 관리하는 Dictionary 변수 존재
- Dictionary의 key,value 들은 해당 클래스에서 생성시 BaseScene을 찾아 넣어주도록 설계
- Key : NetIdentity Net Id
- Value : GameObject
- 만약 인스턴스 씬이면 BaseInstanceScene을 상속받은 클래스에서 S\_SceneObject\_Remove 함수를 호출해서 만약 해당 씬에 플레이어가 0이면 실행중인 코루틴을 정지시키고 3초 뒤 씬을 제거하도록 설계

# 3주차 – 인스턴스 던전 및 아이템 구현

[ServerCallback]

```
public IEnumerator DungeonSceneLoadAdditive()
```

```
// 정적 리스트 Terrain 리스트 (모든 BaseSceneNetwork는 하나의 리스트를 사용한다)  
static List<Terrain> terrainList = new List<Terrain>();
```

```
public static void DisableTerrainList()  
{  
    if (Managers.Instance.mode == NetworkManagerMode.Host)  
    {  
        for (int i = 0; i < terrainList.Count; i++)  
        {  
            if (terrainList[i] != null)  
            {  
                terrainList[i].enabled = false;  
            }  
        }  
        terrainList.Clear();  
    }  
}
```

```
// 서버나 호스트에서 해당 씬의 물리 캐싱  
public void MoveToScene(GameObject newPlayer, Scene scene)  
{  
    SceneManager.MoveGameObjectToScene(newPlayer, scene);  
  
    if (newPlayer.TryGetComponent(out Creature creature))  
        creature.GetSetPhysics = scene.GetPhysicsScene();  
}
```

## 던전 씬 로드

- SceneManager함수를 통해 호출이 가능
- 씬 로드 전 Mirror에선 호스트 플레이어는 Terrain 오브젝트가 비활성화 되지 않아 각 씬에 존재하는 BaseSceneNetwork 을 상속받은 클래스에서 미리 terrainList에 terrain오브젝트를 넣어주고 씬 로드시 DisableTerrainList를 호출하여 수동으로 이전 씬의 Terrain을 비활성화 시키도록 코드를 작성하였습니다.
- 해당 씬이 로드가 완료되면 MoveToScene을 통해 해당 씬을 활성화 시키고 Creature클래스를 상속받은 오브젝트에 PhysicsScene을 캐싱

## 4주차 – 부족한 콘텐츠 보강

- 플레이어블 캐릭터 추가 및 스킬 구현, 사운드 추가, 이펙트 추가 버그 수정
- 월드 아이템 구현



# 4주차 – 부족한 콘텐츠 보강

## BaseSceneNetwork 사운드 및 이펙트 호출

- 매니저 컴포넌트를 가지고 있는 오브젝트는 NetId가 없기 때문에 RPC를 하면 오류가 발생하기에 NetworkIdentity를 가지고 있는 BaseSceneNetwork에서 함수 구현하여 사운드 및 이펙트 구현

```
public void RPC_Play2D(string path, Define.Sound2D type, float pitch)
{
    #if UNITY_SERVER
        if (Util.IsSameScene(gameObject) == false)
            return;

        Managers.Sound.Play2D(path, type, pitch);
    #endif
}

[ClientRpc]
public void RPC_Play3D(string path, Vector3 pos, Define.Sound3D type, float volume, float pitch, float minDistance, float maxDistance)
{
    #if UNITY_SERVER
        if (Util.IsSameScene(gameObject) == false)
            return;

        Managers.Sound.Play3D(path, pos, type, volume, pitch, minDistance, maxDistance);
    #endif
}

[ClientRpc]
public void RPC_Play3D(string path, GameObject gameObject, bool isAttach, Define.Sound3D type, float volume, float pitch, float minDistance, float maxDistance)
{
    #if UNITY_SERVER
        if (Util.IsSameScene(gameObject) == false)
            return;

        if (gameObject != null)
            Managers.Sound.Play3D(path, gameObject.transform, isAttach, type, volume, pitch, minDistance, maxDistance);
    #endif
}

[ClientRpc]
public void RPC_Effect(string key, Vector3 spawnPos, Quaternion qut)
{
    #if UNITY_SERVER
        if (Util.IsSameScene(gameObject) == false)
            return;

        GameObject effectObj = Managers.Resource.Instantiate(key, null, true);
        effectObj.transform.parent = null;
        effectObj.transform.position = spawnPos;
        effectObj.transform.rotation = qut;

        if (effectObj.TryGetComponent(out IEffectPlay effectPlay))
        {
            effectPlay.EffectPlay();
        }
    #endif
}
```

# 4주차 – 부족한 콘텐츠 보강

```
public class EffectLifeTime : MonoBehaviour, IEffectPlay
{
    protected ParticleSystem _particleSystem;

    protected void Awake()
    {
        TryGetComponent(out _particleSystem);
    }

    public virtual void EffectPlay()
    {
        Invoke(nameof(PushPool), _particleSystem.main.duration + 0.1f);
    }

    protected void PushPool()
    {
        Managers.Resource.Destroy(gameObject);
    }
}
```

```
public class EffectManualLifeTime : EffectLifeTime
{
    [SerializeField]
    protected float effectDuration;

    public override void EffectPlay()
    {
        Invoke(nameof(PushPool), effectDuration + 0.1f);
    }
}
```

## 이펙트 클래스

- Prefab으로 구현된 이펙트 오브젝트에 Particle컴포넌트의 main.duration 값에 0.1f 초를 더한 후 오브젝트를 제거하도록 구현
- 만약 사용자 지정으로 duration을 지정하고 싶으면 EffectLifeTime대신 EffectManualLifeTime 클래스 컴포넌트 추가 하여 멤버 변수 effectDuration 값을 설정하여 지속시간 설정

# 개발에 사용된 에셋

- ATART Character Bundle
- CartoonFX Rematser
- FANTASTIC - City Pack
- FANTASTIC UI STARTER PACK
- Fantasy Adventure Environment
- Footsteps Sound Pack
- FREE Casual Game SFX Pack
- Game Music Pack - SUITE
- Mirror
- Monsters Forest Pack
- RPG Inventory Icons
- Ultimate Sound FX Bundle
- 100 Special Skills Effects Pack
- etc...

# 구현에 어려웠던 점

- 호스트, 서버, 클라이언트 각 포지션 별로 고려해서 코드 짜기가 어려웠고 복잡했습니다.
- 위와 같이 코드를 작성하다 보니 매번 테스트를 해야 해서 구현하기 어려웠습니다.
- 인벤토리, 아이템 구현과 UI 연동과 같이 복잡한 구조의 클래스를 구현하기 매우 어려웠습니다.
- 호스트 플레이어에서 씬을 이동할씨 이전씬의 Terrain 및 World UI가 비활성화 되지 않는 문제 발생하여 수정하는데 매우 어려웠습니다.

# 해결

- 많은 시간을 투자하여 코드를 작성하고 테스트를 했습니다.
- 블로그, 카페 글을 참고하여 인벤토리, 아이템 클래스를 구현하였습니다.
- 씬을 이동하는 사용자 함수를 구현하여 호스트가 씬을 이동할시 현재 Terrain을 비활성화 시키도록 코드를 수정하였고 World UI는 CanvasGroup의 alpha의 초기값을 0으로 하고 NetworkObjectInfo 클래스의 OnStartClient()에서 alpha값을 1로 수정하고, OnStopClient()일시 alpha값을 0으로 하도록 하여 문제를 해결하였습니다.

# 개발 후 느낀점 혹은 추후 개선 방향

- 네트워크 요소를 넣으니까 구조 설계의 어려움 그리고 코드 버그 테스트 시간이 너무 오래걸렸다.
- FSM에서 해당 상태를 반복해서 변경할 시 유니티 애니메이션이 정상적으로 동작을 안되서 고생을 했다.
- 이전에 포톤에 대해 잘알아서 미래도 쉬울줄 알았지만 기본적인 구조는 거의 동일했지만 미래만의 로직을 이해하는데 조금 시간이 걸렸다.
- 미구현한 상점, 마법사 사운드 등을 구현할 수 있으면 구현할 예정
- 몬스터의 스텝을 디테일하게 수정할 예정